Tecnologias Web – Folha de revisão

Gonçalo Sousa Temas 1–11. As regras e tabelas do resumo, sem exemplos: para rever antes do teste.

HTML

Documento: `<!DOCTYPE html>` (não é anotação nem comentário) → raiz `<html>` com só `<head>` (não mostrado) e `<body>`. **URL** absoluto = protocolo `://` servidor `[:porta]` caminho; relativo = resolvido contra a diretoria da página. **Atributos**: só na anotação de início, ordem irrelevante; aspas obrigatórias com espaços. **Entidades** `< > & " '  `. Brancos colapsam num espaço. Comentários `<!-- -->` não se aninham (sem `--`). Fechos opcionais: `</li>`, `</option>`, `</td>`.

Em linha	Bloco
`b i u s sup sub a br code`	`p h1..h6 hr ul ol li dl dt dd img`

Formulários: `action` (URL), `method` (`get` → dados no URL; `post` → no corpo). `name`=chave, `value`=valor. `enctype`: urlencoded (omissão), `multipart/form-data` (ficheiros, só POST), `text/plain`. `select size=1` menu / `>1` lista. **Tabelas**: `table tr td th`, `colspan`/`rowspan`, `thead tbody tfoot`.

CSS

Cascata: origem (autor > leitor) → especificidade → última declarada; `!important` passa à frente. **Especificidade** (b, c, d) = (#ids, .classes/[atr]/:pseudo-classe, tipos/::pseudo-elemento), compara-se posição a posição; `style=` vence tudo menos `!important`.

Seletor	Significado
`p.x` / `p .x`	`p` com classe `x` / `.x` dentro de `p`
`a > b`	filho direto
`a + b` / `a ~ b`	irmão logo a seguir / qualquer irmão seguinte
`[a~=v]` / `[a^=v]`	`v` entre as palavras / começa por `v`
`:nth-child(2n+1)`	ímpares (`odd`)
`::before`, `::after`	precisam de `content`

Caixa: content → padding → border → margin; largura total = `width` + 2·padding + 2·border + 2·margin (`content-box`). Bordo só com `border-style`. Fundo pinta content+padding.

`position`: `static` (ignora coords), `relative` (desloca, mantém espaço), `absolute` (1.º antepassado posicionado, sai do fluxo), `fixed` (janela), `sticky` (relative → fixed no contentor). `z-index` só em posicionados. `display:none` liberta espaço; `visibility:hidden` mantém. **Centrar texto**: `text-align:center` + `line-height = height`. **Bloco**: `margin:auto` + `relative; top:50%; margin-top:-metade`.

Flexbox: `justify-content` = eixo principal (`row` horizontal), `align-items` = transversal, `align-content` = linhas. Item: `flex: grow shrink basis` (omissão `0 1 auto`), `order`, `align-self`. **Grid**: linhas de grelha começam em 1; `grid-template-columns`, `gap`, `grid-column: span 2`; implícitas → `grid-auto-rows`.

JavaScript

`const` > `let` (bloco) > `var` (função, elevado sem valor). Sem declaração → global. `function` sobe inteira; `let`/`const` antes da linha → erro. ASI: não insere `;` se a linha seguinte continua válida (ex. começa por `(`); `return` sozinho na linha → `return;`. `a && b`/`a || b` devolvem um operando. `==` converte (`1=="1"`, `null==undefined`, `0==""`), `===` não; `NaN` ≠ tudo (`Number.isNaN`). `for..in` chaves, `for..of` valores (Map só `of`). `obj.k` = `obj['k']`. `delete` num array deixa buraco (`splice`). Desconstrução: arrays por posição, objetos por nome (`{a: novo}`).

Classes: `constructor`, campos em `this`, `get/set` (guardar noutro campo), `static` (sem `this`, chamar pela classe), `extends` + `super` (classe imediatamente acima). `this` = objeto à esquerda do ponto; setas não têm `this` próprio → usar em *callbacks* (ou `.bind(this)`). Ciclo com `let` captura um `i` por iteração.

Saída: `document.write` (página), `alert/prompt/confirm` (diálogo: nada/string ou null/booleano), `console.log` (depurar).

DOM e eventos

`getElementById`, `querySelector(All)`, `createElement`, `createTextNode` (soltos até `appendChild/insertBefore(n, ref)`; `ref=null` = fim). `Node`: `parentNode firstChild nextSibling childNodes` (espaços em branco são nós). `Element`: `get/setAttribute className id innerHTML`. CSSOM: `style.fontSize` (camelCase).

Registo: atributo `onclick="..."` → propriedade `el.onclick = f` (1 só, após `window.onload`) → `addEventListener("click", f, fase)` (vários; `true` captura, `false` bubbling). Fases: captura desce, alvo, bubbling sobe. `preventDefault()` ação por omissão; `stopPropagation()`.

HTTP

Pedido: `MÉTODO URL VERSÃO`, cabeçalhos, linha vazia, corpo (EOL CR+LF). Resposta: `VERSÃO CÓDIGO DESCRIÇÃO`. Sem conexão, sem estado, independente de media. URL na linha de pedido sem servidor (exceto *proxy*); `#fragmento` nunca enviado.

Método	Seguro	Idempotente	Cachável
`GET`, `HEAD`	sim	sim	sim
`POST`	não	não	condições
`PUT`, `DELETE`	não	sim	não
`OPTIONS`, `TRACE`	sim	sim	não

Códigos: 1XX info (101 Switching), 2XX (200, 201 Created, 204 No Content), 3XX (301 permanente + `Location`, 302, 303 GET depois de POST, 304 Not Modified, 307 mesmo método), 4XX cliente (400, 401, 403, 404), 5XX servidor (500, 501, 502, 503, 504).

Cabeçalhos: `Set-Cookie` (resposta; `Expires`, `Path`) → `Cookie` (pedido, `;`). Cache: `Cache-control: max-age/no-store/no-cache`, `ETag` → `If-None-Match` → 304. CORS: `Access-Control-Allow-Origin`; *preflight* `OPTIONS` com `Access-Control-Request-Method/Headers` + `Origin` → `Allow-Origin` + `Allow-Methods`. Autenticação: 401 + `WWW-Authenticate: Basic realm` → `Authorization: Basic base64` (só com HTTPS). *urlencoded*: espaço `+`, outros `%XX`, `=` e `&`.

Comunicação assíncrona

XHR: `open(método, url, async)` → `onreadystatechange` (reagir com `readyState == 4`) → `send(corpo)`; `status`, `responseText`; síncrono *deprecated*. **fetch** → `Promise<Response>` (`ok`, `status`, `json()`, `text()` também promessas). `.then` devolve sempre nova promessa; rejeição salta para `.catch`. `async` devolve promessa (`return` → resolve, `throw` → reject); `await` só dentro de `async`. Ordem: síncrono primeiro, `.then` depois (microtasks).

	SSE (`EventSource`)	WebSocket
Sentido	servidor → cliente	bidirecional (`send`)
Protocolo	HTTP, `text/event-stream`, `data: ...\n\n`	`ws://`, *upgrade* → 101
Reconexão	automática	não
Dados	texto	texto e binário

Outras APIs

Web Storage: `localStorage` (para sempre, partilhado por separadores) / `sessionStorage` (separador); `setItem/getItem` guardam *strings* → `JSON.stringify/parse`. **Canvas**: dimensões nos atributos (não CSS); origem canto superior esquerdo, Y para baixo, ângulos em radianos no sentido dos ponteiros. `fillRect/strokeRect/clearRect`; caminho: `beginPath moveTo lineTo closePath stroke fill` (`fill` fecha sempre); `arc(x,y,r,a0,a1,antihorario)`, `quadraticCurveTo`, `bezierCurveTo`. Transformações compõem-se e a ordem importa; `save/restore` em pilha. Animação: `canvas.width = canvas.width` limpa tudo + `requestAnimationFrame` recursivo.

Servidor

Node: uma *thread*, I/O não bloqueante (callbacks). `http.createServer((req,res)=>...) .listen(p)`; `req.method`, `req.url` (relativo); `res.writeHead(estado, cab)`, `write`, `end`. `url.parse(u, true)` → `pathname`, `query`. Corpo: `req.on('data').on('end').on('error')`. `fs.readFile(c, 'utf8', (err,data))`, `fsp` com promessas, `Sync` bloqueia. Módulos: `require` + `module.exports` (função/classe anónima = 1 valor; propriedades = vários).

CGI: um processo por pedido (`fork+exec`); dados em variáveis de ambiente (`REQUEST_METHOD`, `QUERY_STRING`, `CONTENT_LENGTH`, `REQUEST_URI = PATH_INFO + QUERY_STRING`), corpo no `stdin`; resposta no `stdout`: [`Status:`], `Content-type` obrigatório, linha em branco, corpo.